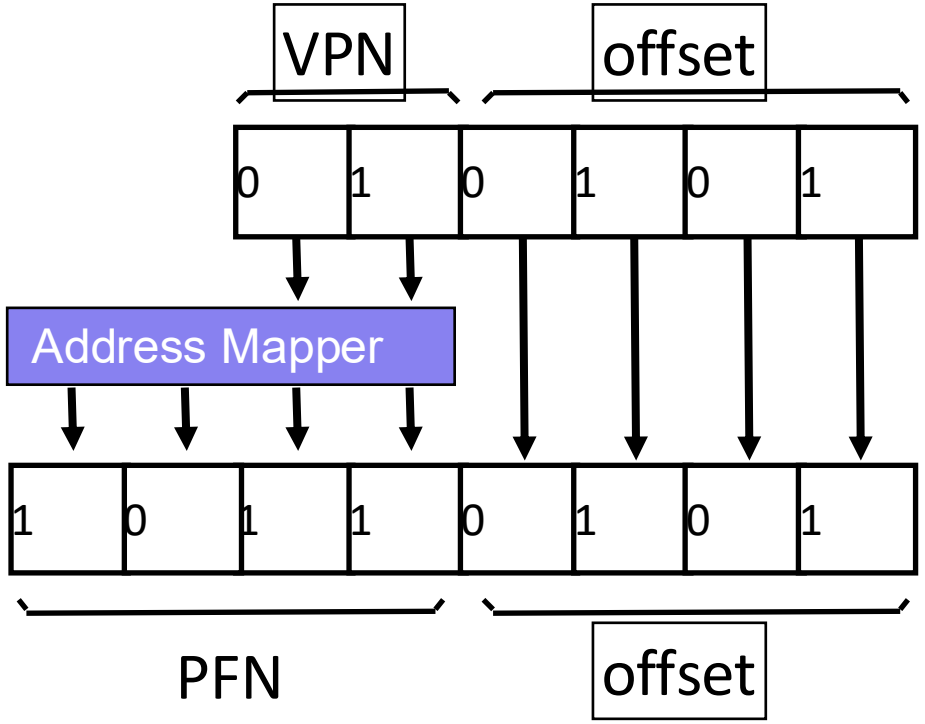
Memory Virtualization



Review: Virt ➔ Phys page mapping

Number of bits in the virtual address not equal the number of bits in the physical address



How should OS translate VPN to PFN?

For segmentation, OS used a formula (e.g., phys addr = virt_offset + base_reg)

For paging, OS needs a different mapping mechanism

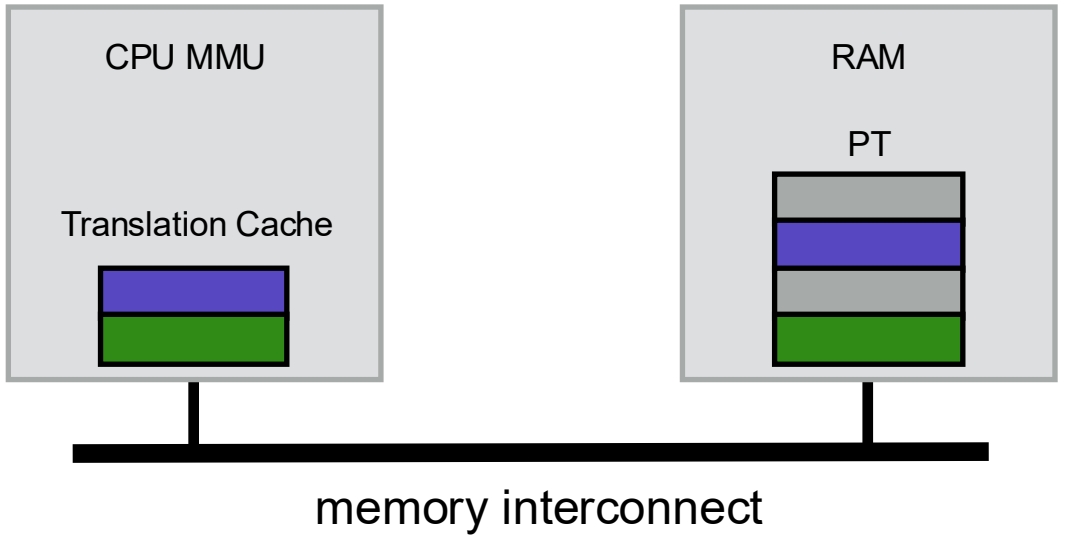
Q: What data structure is good?

Linear Page Table

A simple option: an array

Review: TLBs cache page translations

We couldn't store entire page table in MMU, but we can store a fast cache



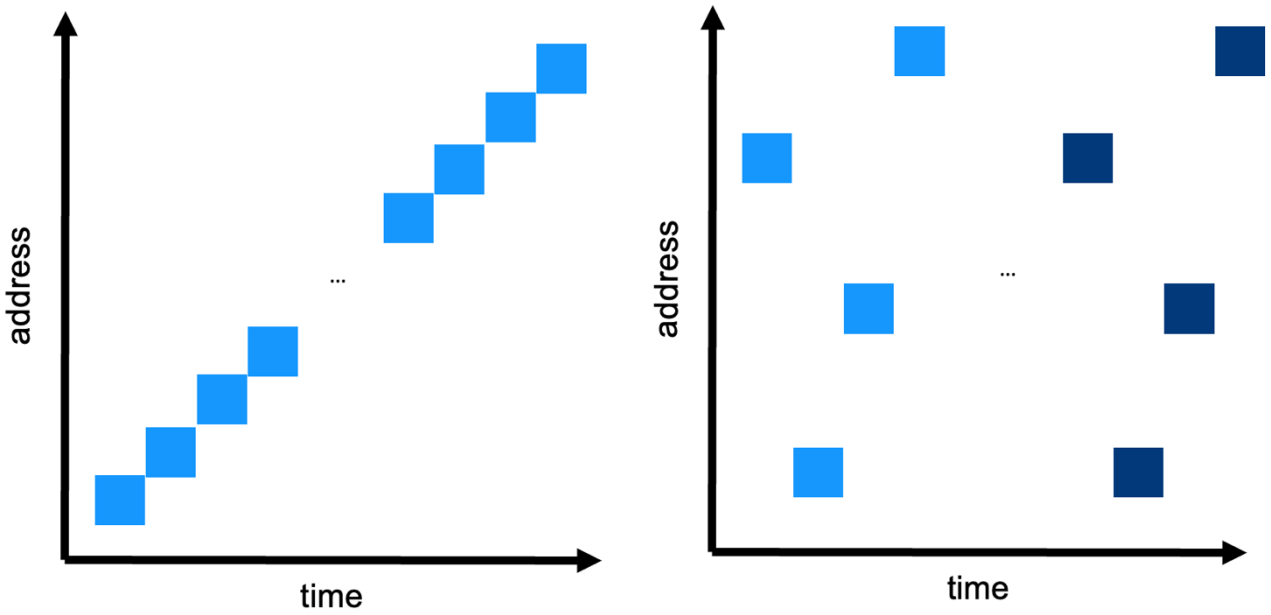
Some popular entries

TLB: **T**ranslation **L**ookaside **B**uffer

Review: Workload Access Patterns

Workload A

```
int sum = 0;
for (i=0; i<2048; i++) {
      sum += a[i];
}
```



Workload B

```
int sum = 0;
srand(1234);
for (i=0; i<1000; i++) {
   sum += a[rand() % N];
}
srand(1234);
for (i=0; i<1000; i++) {
   sum += a[rand() % N];
}
```

Spatial and Temporal locality

Differentiating processes

- So far, we assumed VPNs are unique. But VPNs are not unique across multiple processes. What should we do?
- Option 1: “Flush” TLBs upon every context switch (valid = 0)
  - Problem: poor performance after each context switch
- Option 2: Attach an address space identifier to TLB entry

VPN	PFN	valid	prot
10	100	1	rwx
—	—	0	—
10	170	1	rwx
—	—	0	—

VPN	PFN	valid	prot	ASID
10	100	1	rwx	1
—	—	0	—	—
10	170	1	rwx	2
—	—	0	—	—

A full example with TLBs

On TLB miss: lookups with more paging levels more expensive

How much does a miss cost?

Assume 3-level page table, 256-byte pages, 16-bit addresses
Assume ASID of current process is 211
How many physical accesses for each instruction?

(a) 0xAA10: movl 0x1111, %edi

(b) 0xBB13: addl $0x3, %edi

(c) 0x0519: movl %edi, 0xFF10

ASID	VPN	PFN	Valid
211	0xbb	0x91	1
211	0xff	0x23	1
122	0x05	0x91	1
211	0x05	0x12	0

0xaa: (TLB miss -> 3 for addr trans) + 1 instr fetch
0x11: (TLB miss -> 3 for addr trans) + 1 movl

Total: 8

0xbb: (TLB hit -> 0 for addr trans) + 1 instr fetch from 0x9113

Total: 1

0x05: (TLB miss -> 3 for addr trans) + 1 instr fetch
0xff: (TLB hit -> 0 for addr trans) + 1 movl into 0x2310

Total: 5

Summary: Better page tables

Problem:
Simple linear page tables require too much contiguous memory

Many options for efficiently organizing page tables

If OS traps on TLB miss, OS can use any data structure

- e.g., inverted page tables

If Hardware handles TLB miss, page tables must follow specific data structure that hardware knows how to “walk”

- Multi-level page tables used in x86 architecture
- Each page table must fit within a page

Next Topic: What if desired address spaces do not fit in physical memory?

Expanding Virtual Memory

Questions answered:

- How to run process when not enough physical memory?
- When should a page be moved from disk to memory?
- What page in memory should be replaced?
- How can the LRU page be approximated efficiently?

Motivation

OS goal: Support processes when not enough physical memory

- Single process with very large address space
- Large combined address space of multiple processes

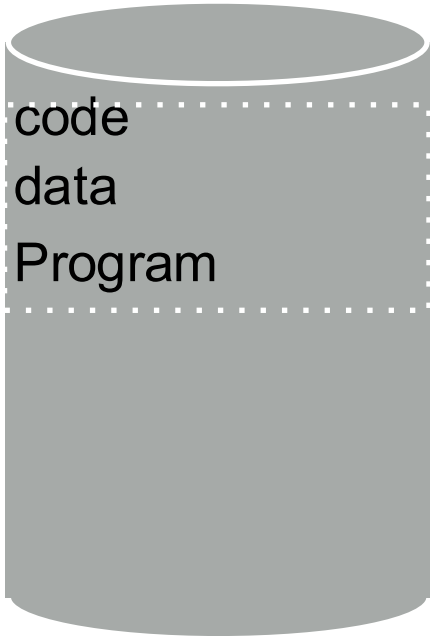
Ideal: Write user code independent of the amount of physical memory

- Correctness, if not performance

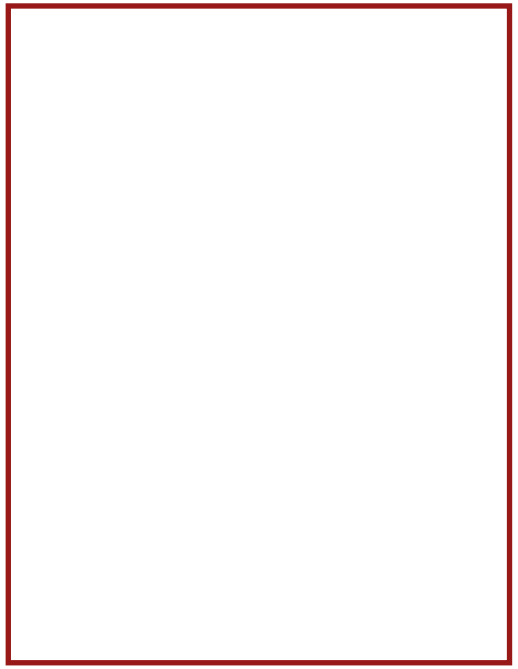
OS can provide an illusion of a large physical memory

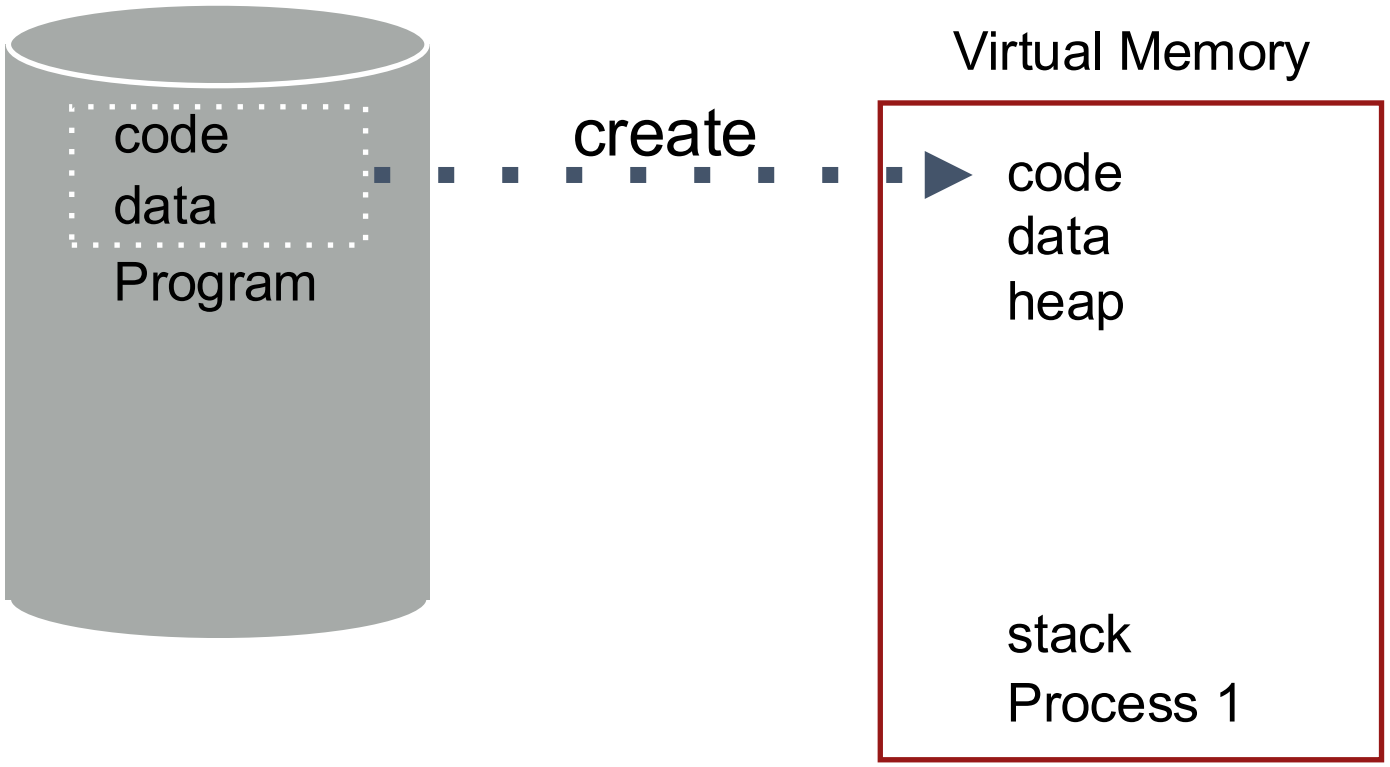
How could we make such an illusion work with good performance?

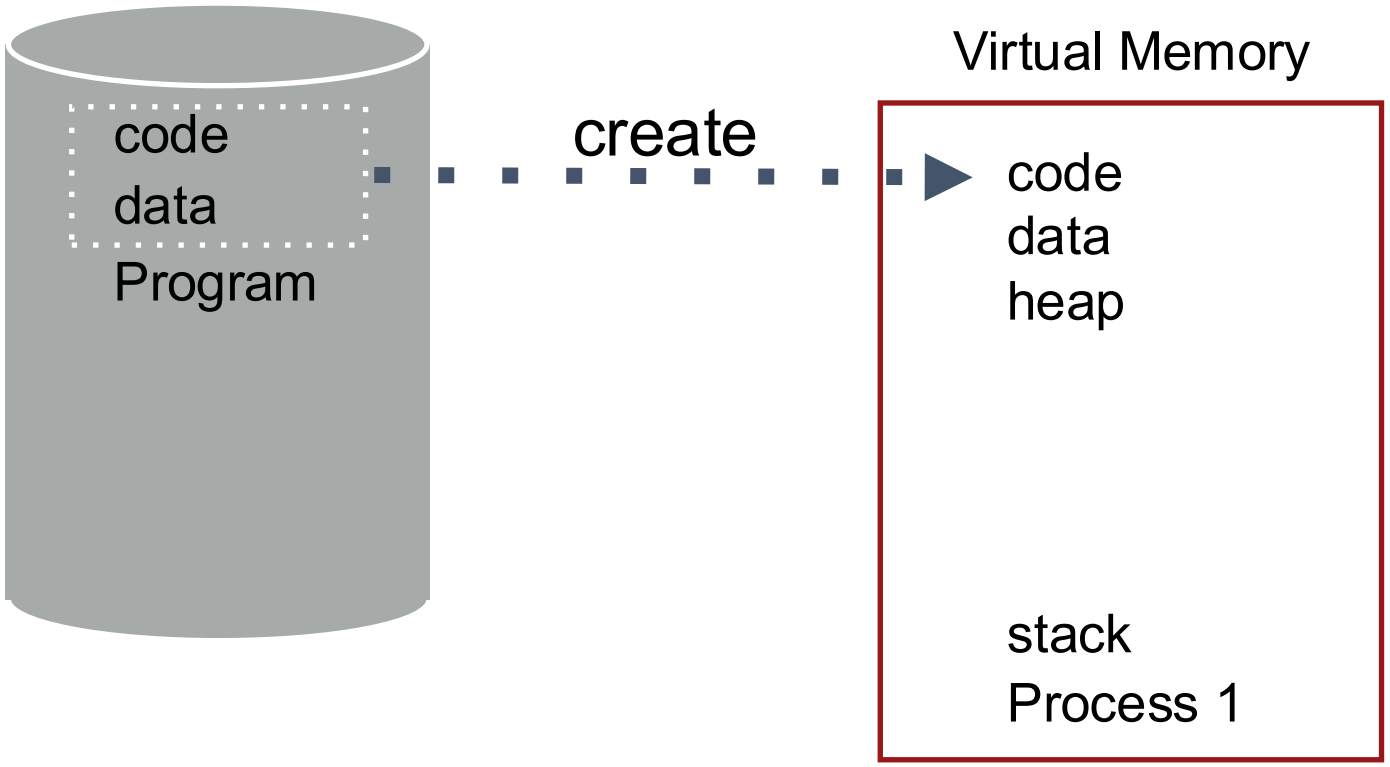
- We rely on the hardware and some typical properties of process execution



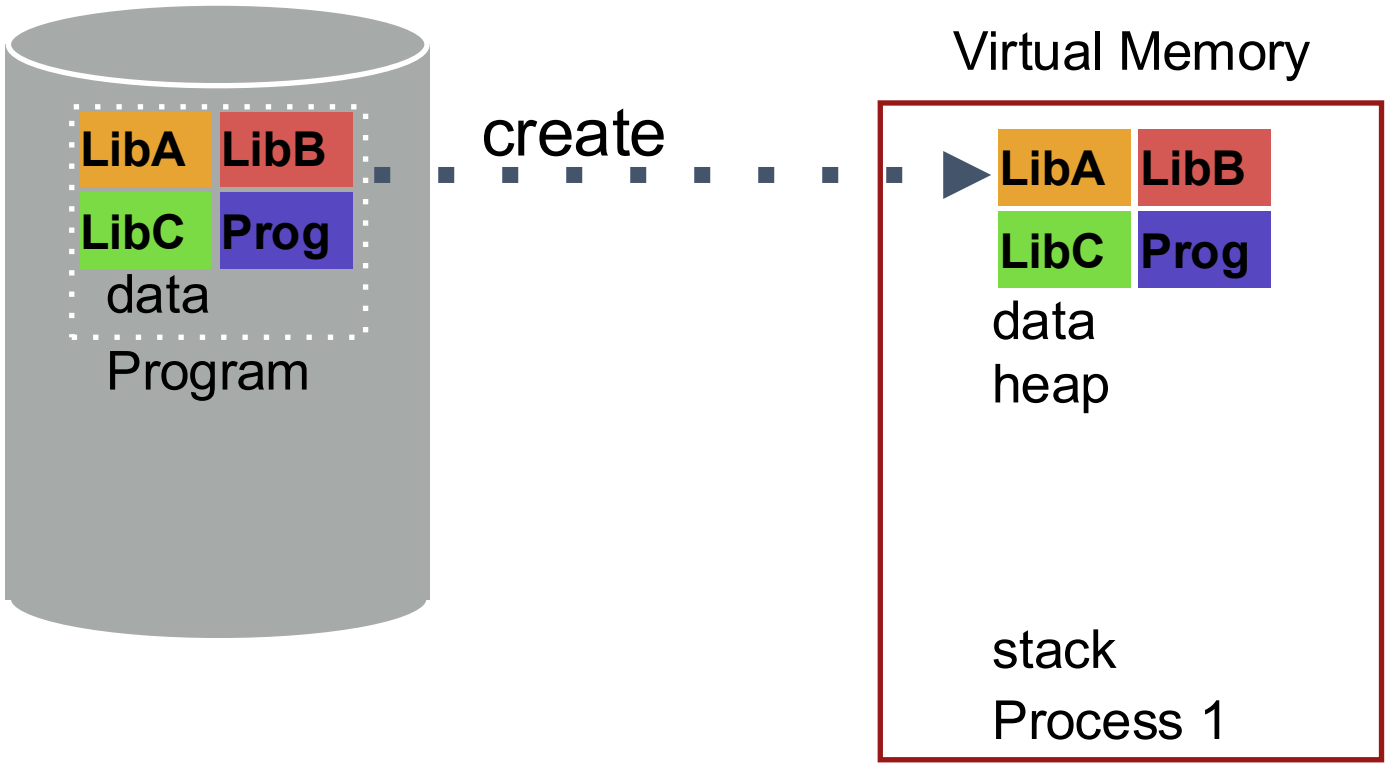
Virtual Memory




code
data
Program
create
Virtual Memory
code
data
heap
stack
Process 1



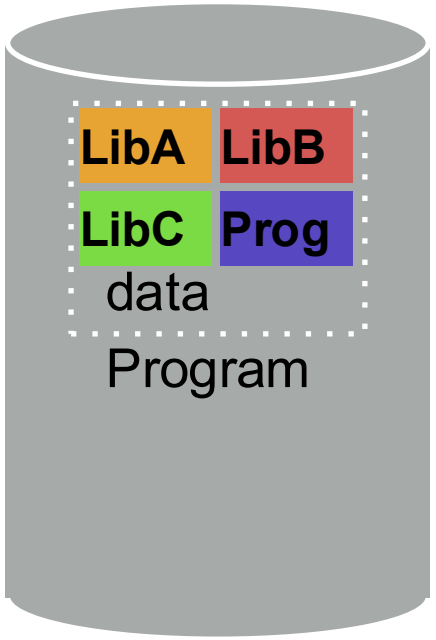
what's in code?



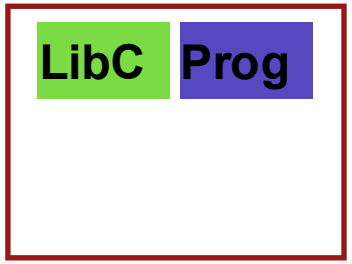
many large libraries, some
of which are rarely/never used

How to avoid wasting physical pages to back rarely used virtual pages?

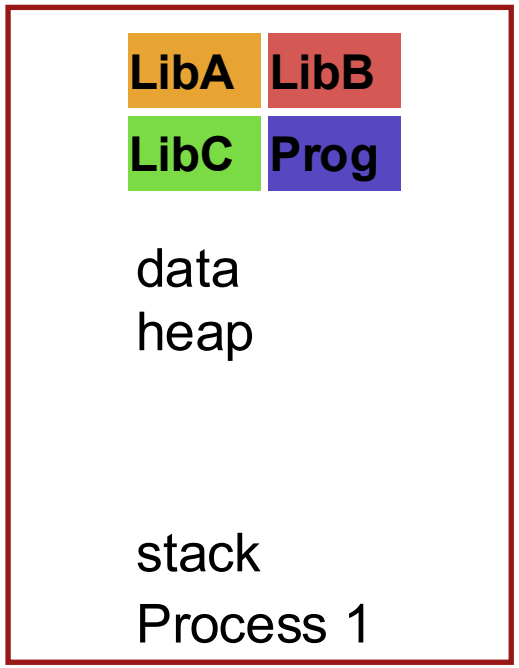
Disk

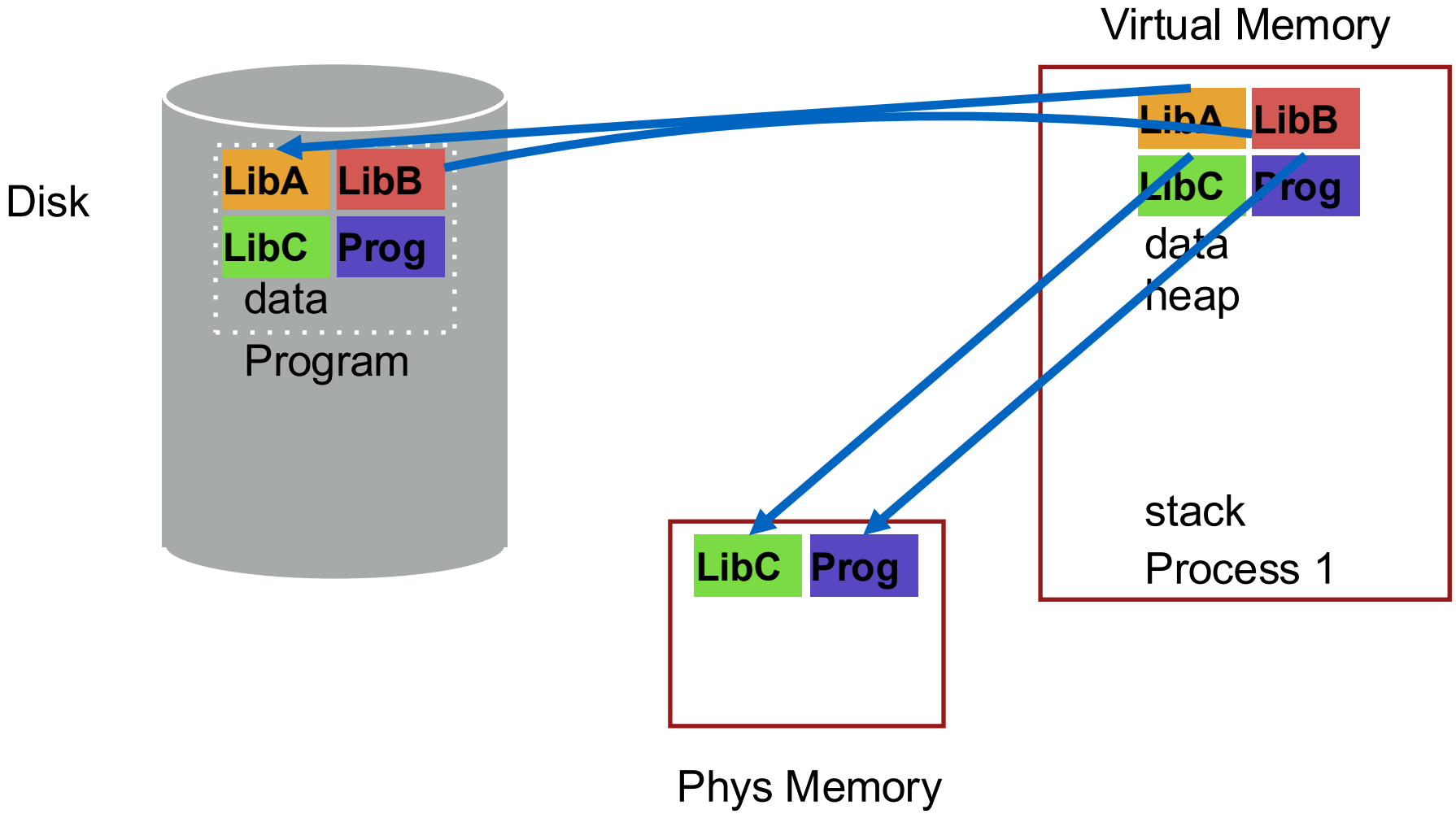


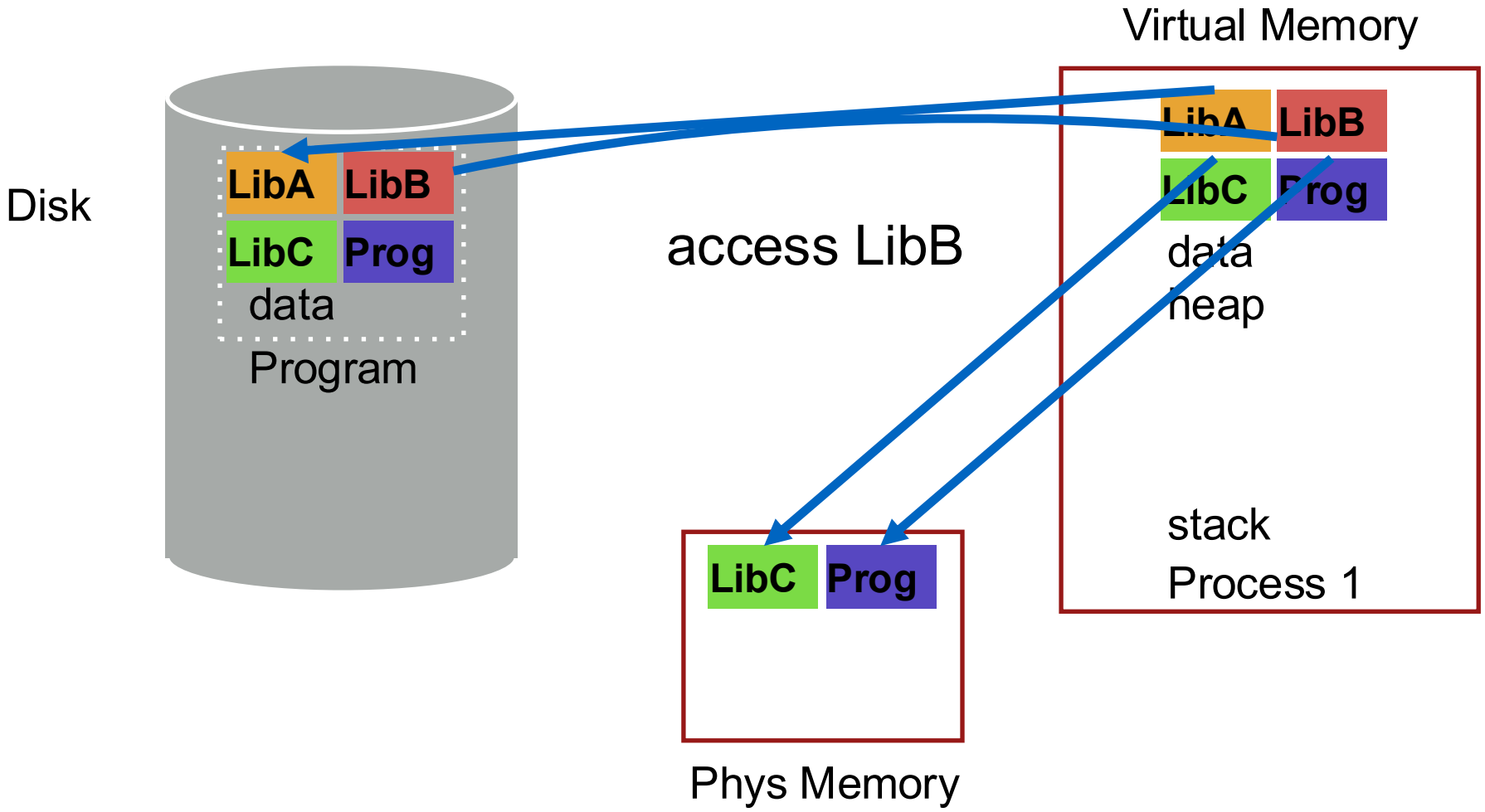
Phys Memory

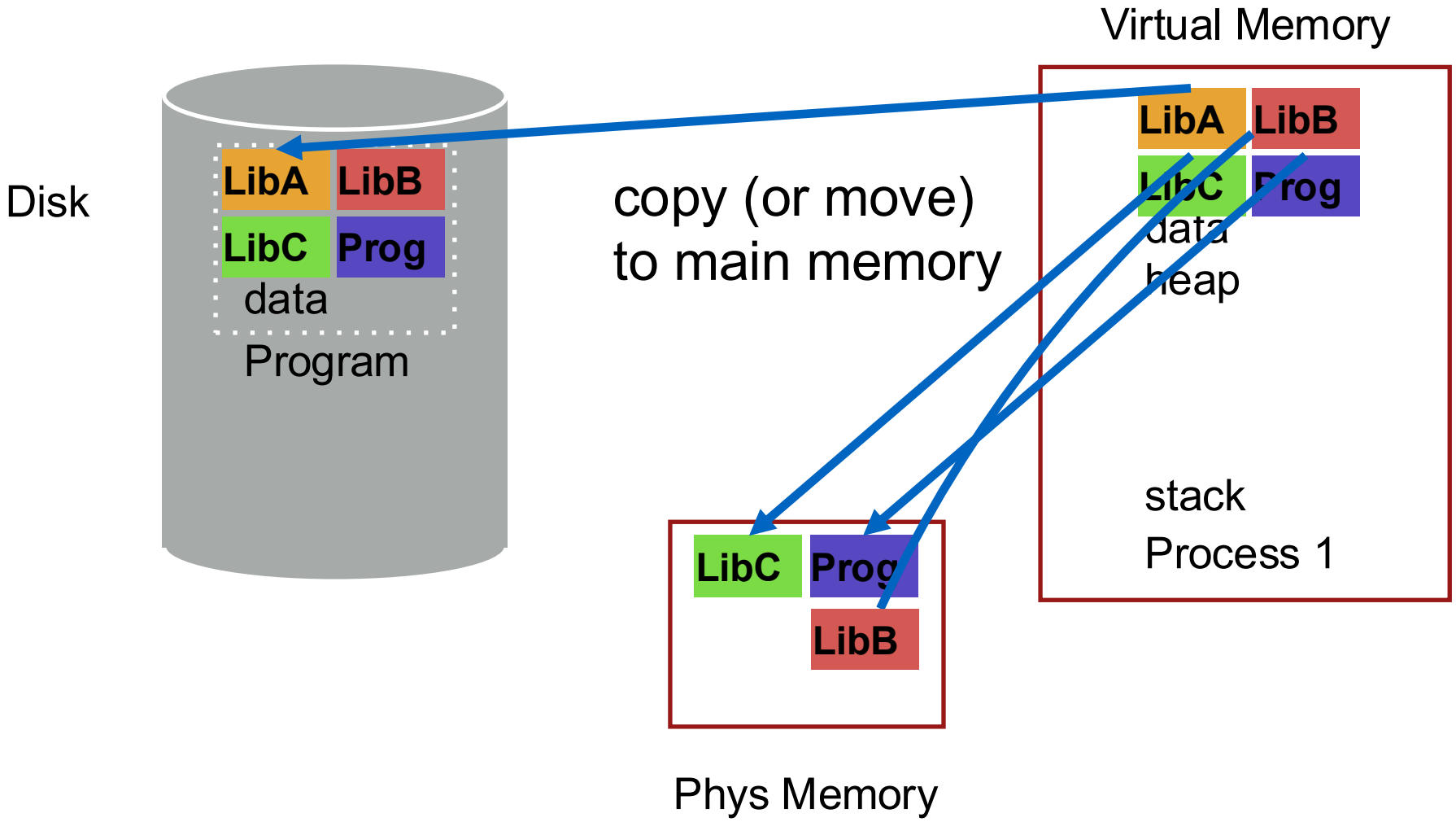


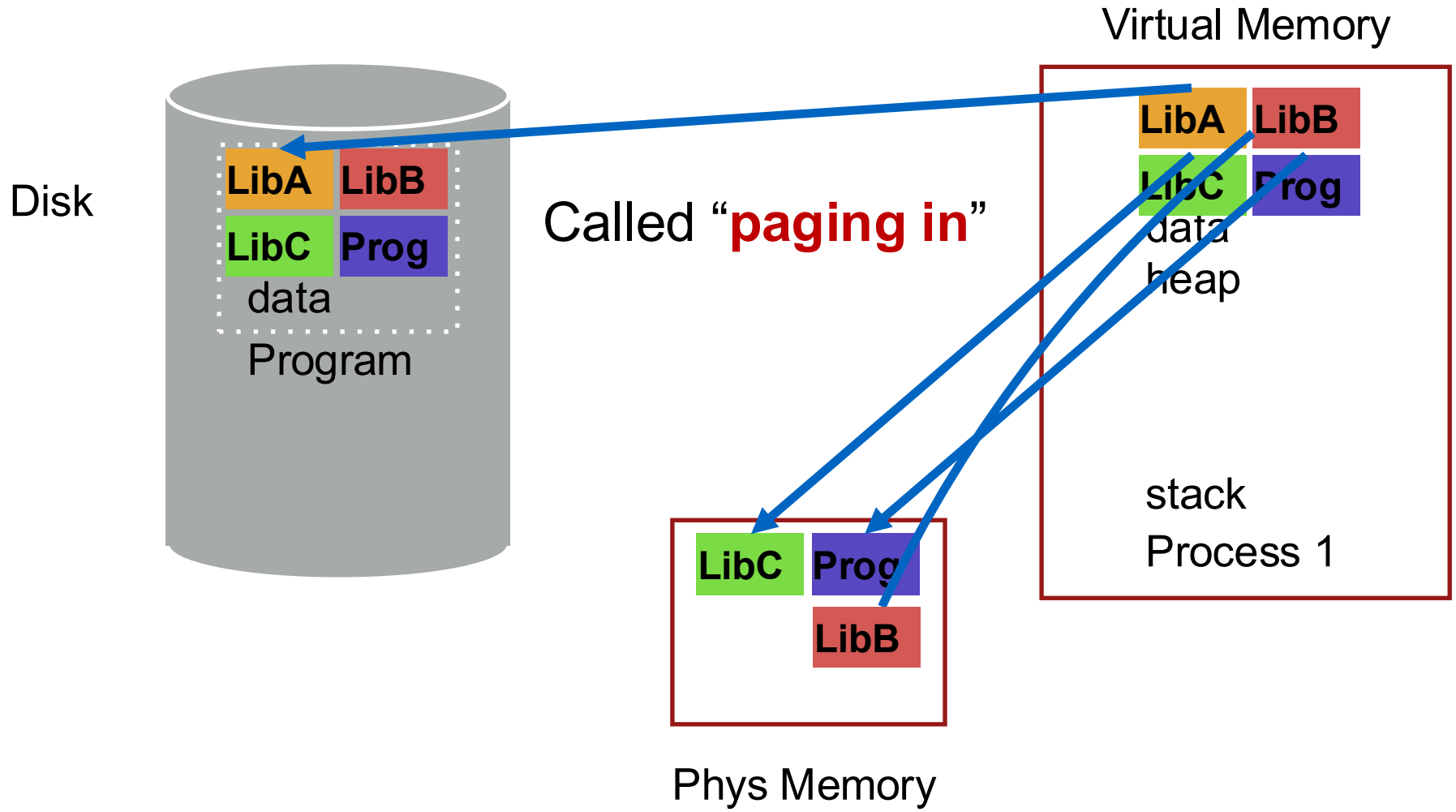
Virtual Memory




Virtual Memory
Disk
LibA
LibB
LibC
Prog
data
Program
LibA
LibB
LibC
Prog
data
heap
stack
Process 1
LibC
Prog
Phys Memory


Virtual Memory
Disk
LibA
LibB
LibC
Prog
data
Program
access LibB
LibA
LibB
LibC
Prog
data
heap
stack
Process 1
LibC
Prog
Phys Memory


Virtual Memory
Disk
LibA
LibB
LibC
Prog
data
Program
copy (or move)
to main memory
LibA
LibB
LibC
Prog
data
heap
stack
Process 1
LibC
Prog
LibB
Phys Memory



(Somewhat confusingly) The idea of maintaining just a limited portion of the virtual address space in physical memory is termed Virtual memory.

Locality of Reference

Effectively: Using main memory as a cache of process virtual memory contents located on disk

Leverage locality of reference within processes

- Spatial: reference memory addresses **near** previously referenced addresses
- Temporal: reference memory addresses that have referenced in the past
- Processes spend majority of time in small portion of code
  - For example: 90% of time in 10% of code

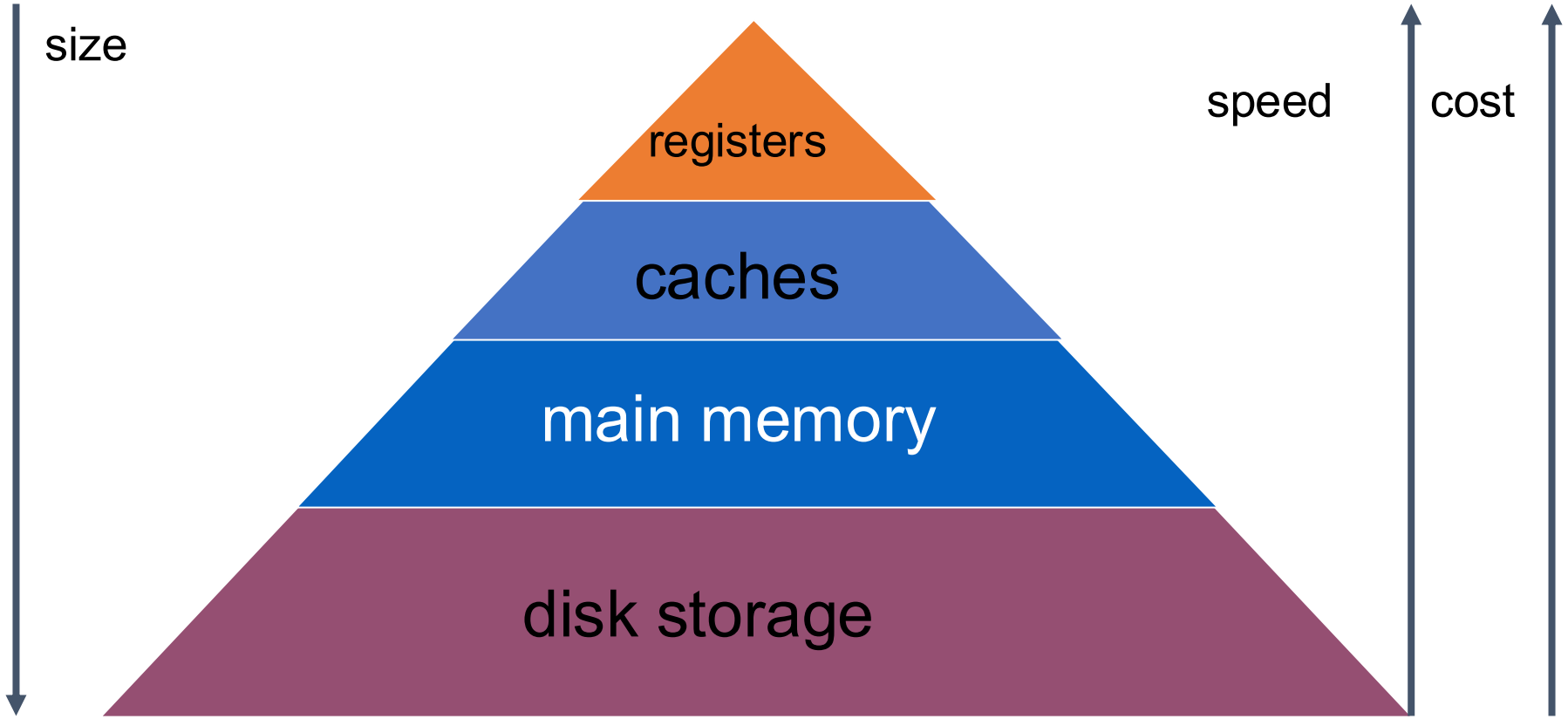
Implication:

- Process only uses small amount of address space at any moment
- Only small amount of address space must be resident in physical memory

Memory Hierarchy

Leverage memory hierarchy of machine architecture

Each layer acts as “backing store” for layer above



Virtual Memory Intuition

Idea: OS keeps unreferenced pages on disk

- Slower, cheaper backing store than memory

Process can run when not all pages are loaded into physical memory, as long as the pages it needs are in physical memory

OS and hardware cooperate to provide illusion of a large, fast physical memory (as large as the disk), by relying on a slower disk

- Same behavior as if all of address space in physical memory
- Hope: provide similar (if not the same) performance

Requirements:

- OS must have a **mechanism** to identify the location of each page in the address space, which can be either in the memory or on disk
- OS must have a **policy** to determine which pages live in memory and on disk

Virtual Address Space Mechanisms

Each page in virtual address space maps to one of three locations:

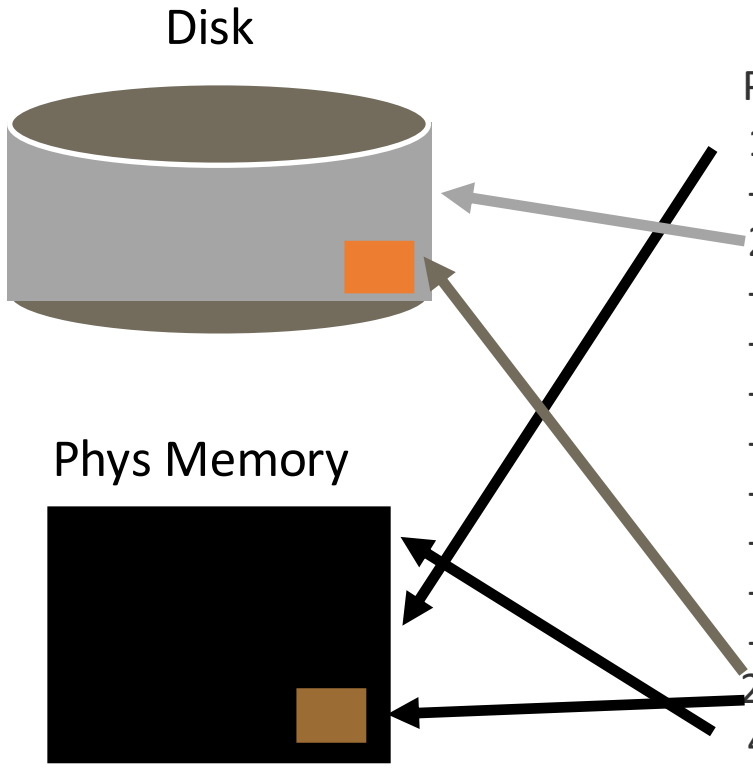
- Physical main memory: Small, fast, expensive
- Disk (backing store): Large, slow, cheap
- Nothing (error): Free, unmapped

Extend page tables with extra information

- `permissions (r/w), valid, present`
- Page in memory: `present` bit set in PTE
- Page on disk: `present` bit cleared

- PTE with cleared present bit points to block on disk (reuse PFN)
  - Causes trap into OS when page is referenced
  - **Trap: page fault**

Present Bit



PFN	valid	prot	present	
10	1	r-x	1	
-	0	-	-	
23	1	rw-	0	
-	0	-	-	
-	0	-	-	
-	0	-	-	
-	0	-	-	
-	0	-	-	
-	0	-	-	
-	0	-	-	
-	0	-	-	
28	1	rw-	1	// VPN 0xb
4	1	rw-	1	

What if access vpn 0xb?